1 Приложение A. Суммы и ряды

A.1. Суммы и их свойства

A.1.1 Найдите простое выражение для $\sum_{k=1}^{n} 2k-1$.

$$\sum_{k=1}^{n} 2k-1 = 2\sum_{k=1}^{n} k - \sum_{k=1}^{n} 1 = n(n-1) - n = n^2.$$

A.1.2 Покажите, используя формулу для гармонического ряда, что $\sum_{k=1}^{n} 1/(2k-1) = ln\sqrt{n} + O(1)$.

$$H_{2n} = \sum_{k=1}^{n} \frac{1}{(2k-1)} + \sum_{k=1}^{n} \frac{1}{2k}$$

$$\sum_{k=1}^{n} \frac{1}{k} = H_n$$

$$\sum_{k=1}^{n} \frac{1}{2k-1} = H_{2n} - \frac{1}{2}H_n$$

$$H_n = \ln n + O(1) \sum_{k=1}^{n} \frac{1}{2k-1} = \ln 2n + O(1) - 1/2(\ln n + O(1)) =$$

$$\ln 2 + \ln n - 1/2(\ln n) + O(1) + 1/2O(1) = \ln n - 1/2(\ln n) + O(1) =$$

$$\ln n * (1 - 1/2) + O(1) = \frac{1}{2}\ln n + O(1) = \ln\sqrt{n} + O(1)$$

A.1.3 Покажите, что для $|x| < 1$ справедливо сотношение $\sum_{k=0}^{\infty} k^2 x^k = x(1+x)/(1-x)^3$.

$$|x| < 1$$
$$\left(\sum_{k=0}^{\infty} x^k\right)' * x = \left(\frac{1}{1-x}\right)' * x$$
$$\left(\sum_{k=0}^{\infty} kx^k\right)' * x = \left(\frac{x}{(1-x)^2}\right)' * x$$
$$\sum_{k=0}^{\infty} k^2 x^k = \left(\frac{(1-x)^2 - 2x(1-x)(-1)}{(1-x)^4}\right) * x$$
$$= \left(\frac{(1-x)^2 + 2x(1-x)}{(1-x)^4}\right) * x =$$
$$\left(\frac{(x+1)}{(1-x)^3}\right) * x = \frac{x(x+1)}{(1-x)^3}$$

A.1.4

A.1.5

A.1.6 Доказать: $\sum_{k=1}^{n} O(f_k(i)) = O(\sum_{k=1}^{n} f_k(i))$

$$\sum_{k=1}^{n} \Theta(f_k(i)) = \Theta(\sum_{k=1}^{n} f_k(i)) \to \sum_{k=1}^{n} O(f_k(i)) = O(\sum_{k=1}^{n} f_k(i))$$

Не полное, однако в главе 3 описано что если $\Theta(f(n)) \to O(f(n))$ Можно привести док-во через определение $O(f(n))$

A.1.7 Вычислите произведение $\prod_{k=1}^{n} 2 * 4^k$.

$$\lg\left(\prod_{k=1}^{n} 2 * 4^k\right) = \sum_{k=1}^{n} \lg 2 * 4^k$$
$$\sum_{k=1}^{n} (\lg 2 + k \lg 4) = n \lg 2 + \lg 4 * \sum_{k=1}^{n} (k) =$$
$$n \lg 2 + \lg 4 \frac{n(n+1)}{2} =$$
$$n \lg 2 + \lg 2n(n+1) = \lg 2n(n+2) = (n^2 + 2n) \lg 2$$

A.1.8 Вычислите произведение $\prod_{k=2}^{n}(1-1/k^2)$.

$$\prod_{k=2}^{n}\frac{k^2-1}{k^2}=$$

$$\prod_{k=2}^{n}\frac{(k-1)*(k+1)}{k^2}=$$

$$\prod_{k=2}^{n}\frac{k-1}{k}*\prod_{k=2}^{n}\frac{k+1}{k}$$

$$\prod_{k=2}^{n}\frac{k-1}{k}=\frac{1}{2}*\frac{2}{3}*\frac{3}{4}*\cdots*\frac{n-1}{n}=\frac{1}{n}$$

$$\prod_{k=2}^{n}\frac{k+1}{k}=\frac{3}{2}*\frac{4}{3}*\frac{5}{4}*\cdots*\frac{n+1}{n}=\frac{n+1}{2}$$

$$\prod_{k=2}^{n}(1-1/k^2)=\frac{1}{n}*\frac{n+1}{2}=\frac{n+1}{2n}$$

A.2. Оценка сумм

A.2.1 Покажите, что сумма $\sum_{k=1}^{n}1/k^2$ ограничена сверху константой.

$$\sum_{k=1}^{n}1/k^2\le c$$

$$\sum_{k=1}^{n}1/k^2=1+\sum_{k=2}^{n}1/k^2$$

$$\sum_{k=2}^{n}1/k^2\le\int_1^n 1/x^2=1-\frac{1}{n}$$

$$\sum_{k=1}^{n}1/k^2\le 1+1-\frac{1}{n}=2-\frac{1}{n}$$

$$\sum_{k=1}^{n}1/k^2\le 2$$

A.2.2 Покажите асимптотическую верхнюю границу суммы.

$$\sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil$$

$$\sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil \leq \sum_{k=0}^{\lfloor \lg n \rfloor} n/2^k + 1 = \sum_{k=0}^{\lfloor \lg n \rfloor} n/2^k + \sum_{k=0}^{\lfloor \lg n \rfloor} 1$$

$$a_1/a_0 = \frac{n * 2^k}{2^{k+1} * n} = \frac{1}{2} = r$$

$$\sum_{k=0}^{\lfloor \lg n \rfloor} n/2^k \leq a_0 * \frac{1}{1-r} = 2n \sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil \leq 2n + \lg n = O(n)$$

A.2.3 Применив разбиение ряда, покажите, что n-е гармоническое число представляет собой $\Omega(\lg n)$.

$$\sum_{k=1}^{n} \frac{1}{k} \geq \sum_{i=0}^{\lfloor \lg n \rfloor - 1} \sum_{j=0}^{2^i - 1} \frac{1}{2^i + j} \geq \sum_{i=0}^{\lfloor \lg n \rfloor - 1} \sum_{j=0}^{2^i - 1} \frac{1}{2^{i+1}} =$$

$$= \sum_{i=0}^{\lfloor \lg n \rfloor - 1} \frac{1}{2} \geq \frac{\lg n - 2}{2} = \Omega(\lg n)$$

A.2.4 Найдите приближенное значение $\sum_{k=1}^{n} k^3$ с помощью интегралов.

$\sum_{k=1}^{n} k^3$ - функция монотонно возрастающая

$$\int_0^n x^3 dx \leq \sum_{k=1}^{n} k^3 \leq \int_1^{n+1} x^3 dx$$

$$\int_0^n x^3 dx = \frac{n^4}{4}$$

$$\int_1^{n+1} x^3 dx = \frac{(n+1)^4}{4} - \frac{1}{4}$$

$$\frac{n^4}{4} \leq \sum_{k=1}^{n} k^3 \leq \frac{(n+1)^4}{4} - \frac{1}{4}$$

$$\sum_{k=1}^{n} k^3 = \Theta(n^4)$$

A.2.5 Почему для получения n-го гармонического числа мы не можем применить интегральное приближение непосредственно к сумме $\sum_{k=1}^{n} \frac{1}{k}$?

$\sum_{k=1}^{n} \frac{1}{k}$ - монотонно убывающая функция, тогда:

$$\int_{1}^{n+1} \frac{1}{x} dx \leq \sum_{k=1}^{n} \frac{1}{k} \leq \int_{0}^{n} \frac{1}{x} dx$$

Интеграл $\int_0^n \frac{1}{x} dx$ является несобственным, равен бесконечности. Следовательно, мы не можем оценить исследуемую функцию, используя интегральное приближение напрямую.

Задачи

A.1. Оценка сумм Дайте асимптотически точные оценки приведённых ниже сумм. Считаем, что $r \geq 0$ и $r \geq 0$ представляют собой константы

а) $\sum_{k=1}^{n} k^r$

$$\int_{0}^{n} x^r dx \leq \sum_{k=1}^{n} k^r \leq \int_{1}^{n+1} x^r dx$$

$$\int_{0}^{n} x^r dx = \frac{n^{r+1}}{r+1}$$

$$\int_{1}^{n+1} x^r dx = \frac{(n+1)^{r+1}}{r+1} - \frac{1}{r+1}$$

$$\frac{n^{r+1}}{r+1} \leq \sum_{k=1}^{n} k^r \leq \frac{(n+1)^{r+1}}{r+1} - \frac{1}{r+1}$$

$$\sum_{k=1}^{n} k^r = \Theta(n^{r+1})$$

б)$\sum_{k=1}^{n} (\lg k)^s$

$$\sum_{k=1}^{n} (\lg k)^s = (\lg 1)^s + \sum_{k=2}^{n} (\lg k)^s = 0 + \sum_{k=2}^{n} (\lg k)^s$$

$$\int_{1}^{n} (\lg x)^s \leq \sum_{k=2}^{n} (\lg k)^s \leq \int_{2}^{n+1} (\lg x)^s$$

$$int_1^n (\lg x)^s$$

в)$\sum_{k=1}^{n} k^r (\lg k)^s$

2 Приложение Б. Множества и прочие художества

Б.1. Множества

Б.1.1 Лень, картинки надо вставлять.

Б.1.2 Докажите обощение законов де Моргана для любого конечно набора множеств:

$$\overline{A_1 \cap A_2 \cap \ldots A_n} = \overline{A_1} \cup \overline{A_2} \cup \ldots \overline{A_n}$$
$$\overline{A_1 \cup A_2 \cup \ldots A_n} = \overline{A_1} \cap \overline{A_2} \cap \ldots \overline{A_n}$$

Ответ:

Б.1.4 Покажите, что множество нечетных натуральных чисел натуральных чисел счетно.

Ответ:

Определим множество все нечётных числе:

$$X = \{x : x \in N \text{ и } x \bmod 2 = 1\}$$

Зададим биекцию $f : N \to X$:

$$f(n) = 2n - 1$$

Таким образом множество X счётно.

Б.1.5 Покажите, что для любого конечного множества S степенное множество 2^S содержит $2^{\|S\|}$ элементов (т.е. имеется $2^{\|S\|}$ различных подмножеств множества S).

Ответ:

По определению 2^S содержит все множества включая пустое множество и само множество S. Допустим $\|S\| = N$, тогда: Общее кол-во множеств можно выразить:

0. Пустое множество - $\binom{N}{0}$

1. Одноэлементных - N или $\binom{N}{1}$;

2. Двух элементных - $\binom{N}{2}$;
3. Трех элементных - $\binom{N}{3}$;
и тд.
Общая сумма:

$$2^{\|S\|} = \sum_{i=0}^{N} \binom{N}{i} = 2^N = 2^{\|S\|}$$

Б.1.6 Дайте индуктивное определение кортежа из n элементов, используя понятие упорядоченное множество.

Ответ:

Рассмотрим базу индукции:

$$(a, b) = \{\{a\}, \{a, b\}\}$$

Выразим данное выражение как:

$$(a, b) = ((a), b) = \{\{a\}, \{a, b\}\}$$

Совершим индуктивный переход, тогда:

$$(a_1, a_2, \cdots a_n, a_{n+1}) = ((a_1, a_2, \cdots a_n), a_{n+1})$$

Б.2. Отношения

Б.2.1

3 Глава 2. Приступаем к изучению

2.1. Сортирока вставкой

2.1.1 Картинка* не знаю как вставить** да и готовить её надо, лень

2.1.2 Перепишите процедуру Insertion-Sort для сортировки в невозрастающем порядке вместо неубывающего.

```
Insertion-Sort(A)
  for j = 2 to A.length
    key = A[j]
    i = j - 1
    while i > 0 and A[i] < key
      A[i+1] = A[i]
      i = i - 1
    A[i+1] = key
```

2.1.3 Рассмотрите **задачу поиска.**

Вход. Последовательность из n чисел $A = \langle a_1, a_2, \ldots, a_n \rangle$ и значение υ.

Выход. Индекс i, такой, что $\upsilon = A[i]$, или специальное значение NIL, если υ в A отсутствует.

Составьте псевдокод **линейного поиска**, при работе которого выполняется сканирование последовательности в поисках значения υ. Докажите корректность алгоритма с помощью инварианта цикла. Убедитесь, что выбранный инвариант цикла удовлетворяет трём необходимым условиям.

```
Linear-Search(A, υ)
  for i = 1 to A.length
    if A[i] == υ
      return i
  return NILL
```

Инвариант цикла:
В начале каждой итерации цикла for подмассив $A[1 \ldots i-1]$ не содержит значение υ.

Инициализация:
При $i = 1$ подмассив $A[1 \ldots 0]$ пуст, поэтому он не содержит υ.

Сохранение:
Предположим, что инвариант верен перед началом итерации с текущим i. Проверяем элемент $A[i]$:

- Если $A[i] = \upsilon$, алгоритм завершается, возвращая i. Перед завершением инвариант выполнен, так как $A[1 \ldots i-1]$ не содержит υ.
- Если $A[i] \neq \upsilon$, то после увеличения i на 1 подмассив $A[1 \ldots i-1]$ (в следующей итерации) по-прежнему не содержит υ, так как к предыдущим элементам добавился $A[i]$, который не равен υ.

Завершение:
Цикл завершается в одном из двух случаев:

1. Найден индекс i, такой что $A[i] = v$. Тогда инвариант гарантирует, что в $A[1 \dots i-1]$ значения v не было.

2. Перебраны все элементы ($i = n+1$). Тогда инвариант означает, что ни один элемент массива не равен v, и алгоритм возвращает NIL.

2.1.4 Рассмотрим задачу сложения двух n-битовых двоичных целых чисел, хранящихся в n-элементных массивах A и B. Сумму этих двух чисел необходимо занести в двоичной форме в (n+1)-элементном массиве C. Приведите строгую формулировку задачи и составьте псевдокод для сложения этих двух чисел.

Формулировка:
Даны два массива $A[1..n]$ и $B[1..n]$, каждый из которых содержит двоичные цифры (0 или 1), представляющие два n-битовых двоичных числа:

$$A = a_1 a_2 \dots a_n, \quad B = b_1 b_2 \dots b_n,$$

где a_1 и b_1 — младшие биты. Требуется вычислить их сумму $C = A + B$, которая является $(n+1)$-битовым двоичным числом, и записать результат в массив $C[1..n+1]$.

Псевдокод:
Bitwise-Addition(A, B, C)
  $carry = 0$
  for $i = 1$ to A.length
    $sum = A[i] + B[i] + carry$
    $C[i] = sum \bmod 2$
    $carry = \lfloor sum/2 \rfloor$
  $C[i+1] = carry$

2.2. Анализ адгоритмов

2.2.1 Выразите функцию $n^3/1000 - 100n^2 - 100n + 3$ в Θ-обозначениях.

$$n^3/1000 - 100n^2 - 100n + 3 = \Theta(n^3)$$

2.2.2 Рассмотрим сортировку элементов массива A, которая выполняется следующим образом. Сначала определяется наименьший элемент массива A, который становится на место $A[1]$. Затем производится поиск второго наименьшего элемента массива A, который становится на место элемента $A[2]$. Этот процесс продолжается для первых $n-1$ элементов массива A. Запишите псевдокод этого алгоритма, известного как сортировка выбором (selection sort). Какой инвариант цикла сохраняется для этого алгоритма? Почему достаточно выполнить для первых $n-1$? Определите время работы алгоритма в наилучшем и наихудшем случаях и запишите его в Θ-обозначениях.

```
Selection-Sort(A)
  for i = 1 to A.length − 1
    minIndex = i
    for j = i + 1 to A.length
      if A[minIndex] > A[j]
        minIndex = j
    swap(A[i], A[minIndex])
```

Инвариант цикла: В начале каждой итерации внешнего цикла for подмассив $A[1 \dots i-1]$ содержит $i-1$ наименьших элементов массива $A[1 \dots n]$, отсортированных по возрастанию.

Почему достаточно выполнить для первых $n-1$? После $n-1$ итераций внешнего цикла первые $n-1$ элементов массива отсортированы и содержат $n-1$ наименьших элементов. Следовательно, последний элемент $A[n]$ автоматически является наибольшим, и весь массив $A[1 \dots n]$ оказывается отсортированным.

Время работы алгоритма: Количество сравнений во внутреннем цикле не зависит от исходного порядка элементов, поэтому время работы одинаково в наилучшем и наихудшем случаях. Общее количество сравнений:

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2} = \Theta(n^2).$$

Таким образом, время работы сортировки выбором составляет $\Theta(n^2)$ в обоих случаях.

2.2.3 Вновь обратимся к алгоритму линейного поиска (см. упр. 2.1.3). Для скольких элементов последовательности в среднем нужно произве-

сти проверку, если предполагается, что все элементы массива с равной вероятностью могут иметь искомое значение? Что происходит в наихудшем случае? Чему равно время работы алгоритма линейного поиска в среднем и в наихудшем случаях в Θ-обозначениях. Обоснуйте ответ.

Ответ:

Предположим, что искомое значение v гарантированно присутствует в массиве $A[1 \dots n]$, и все n позиций равновероятны.

Среднее количество проверок: Если v находится на позиции i, алгоритм выполняет i проверок (сравнений). Математическое ожидание:

$$\frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}.$$

Таким образом, в среднем требуется проверить $\frac{n+1}{2}$ элементов.

Наихудший случай: Наихудший случай наступает, когда v находится на последней позиции ($i = n$). Тогда алгоритм выполняет n проверок.

Время работы в Θ-обозначениях:

$$\text{Средний случай: } \frac{n+1}{2} = \Theta(n), \qquad \text{Наихудший случай: } n = \Theta(n).$$

2.2.4 Каким образом можно модифицировать почти каждый алгоритм, чтобы получить оптимальное время работы в наилучшем случае?

Ответ:

Наилучший случай наступает, когда входные данные уже удовлетворяют инварианту цикла, то есть не требуют преобразований. Для оптимизации достаточно добавить проверку этого условия в начале цикла или внутри него, чтобы при его выполнении алгоритм завершался досрочно.

2.3. Разработка алгоритмов

2.3.1 Лень, надо рисунок делать.

2.3.2 Перепишите процедуру MERGE так, чтобы в ней не использовались сигнальные значения. Сигналом к остановке должен служить тот факт, что все элементы массива L или R скопированны обратно в массив A, после чего в этот массив копируются элементы, оставшиеся в непустом массиве.

```
Merge(A, p, q, r)
  n1 = q - p + 1
  n2 = r - q
  Пусть L[1...n1] и R[1...n2] - новые массивы
  for i = 1 to n1
    L[i] = A[p + i - 1]
  for j = 1 to n2
    R[j] = A[q + j]
  i = 1
  j = 1
  for k = p to r
    if i > n1
      A[k] = R[j]
      j = j + 1
    else if j > n2
      A[k] = L[i]
      i = i + 1
    else if L[i] ≤ R[j]
      A[k] = L[i]
      i = i + 1
    else
      A[k] = R[j]
      j = j + 1
```

2.3.3 Воспользуйтесь методом математической индукции для доказательства того, что, когда n является степенью 2, решением рекуррентного соотношения

$$T(n) = \begin{cases} 2 & \text{если } n = 2 \\ 2T(n/2) + n & \text{если } n = 2^k, k > 1 \end{cases}$$

является $T(n) = n \lg n$.

Доказательство:

База индукции: $n = 2$.

$$T(2) = 2, \quad 2 \lg 2 = 2 \cdot 1 = 2.$$

Таким образом, $T(2) = 2 \lg 2$, база выполняется.

Индукционный переход: Предположим, что для $n = 2^k$, где $k \geq 1$, выполняется $T(2^k) = 2^k \lg(2^k) = 2^k \cdot k$. Докажем, что тогда формула верна для $n = 2^{k+1}$.

По рекуррентному соотношению:

$$T(2^{k+1}) = 2T\left(\frac{2^{k+1}}{2}\right) + 2^{k+1} = 2T(2^k) + 2^{k+1}.$$

Подставим индукционное предположение $T(2^k) = 2^k \cdot k$:

$$T(2^{k+1}) = 2 \cdot (2^k \cdot k) + 2^{k+1} = 2^{k+1} \cdot k + 2^{k+1} = 2^{k+1}(k+1).$$

Заметим, что $\lg(2^{k+1}) = k + 1$, следовательно:

$$T(2^{k+1}) = 2^{k+1} \lg(2^{k+1}) = n \lg n.$$

Индукционный переход доказан.

2.3.4 Сортировку вставками можно представить в виде рекурсивной последовательности. Чтобы отсортировать массив $A[1 \ldots n]$, сначала нужно рекурсивно отсортировать массив $A[1 \ldots n-1]$, после чего в этот отсортированеный массив помещается элемент $A[n]$. Запишите рекурретное уравнение для времени работы этой рекурсивной версии сортировки вставкой.

Рекурретное уравнение:

$$T(n) = \begin{cases} 1 & \text{если } n = 1 \\ T(n-1) + n & \text{если } n > 1 \end{cases}$$

2.3.5 Возвращаясь к задаче поиска (см. упр. 2.1.3), нетрудно заметить, что если последовательность A отсортирована, то можно сравнить значение среднего элемента этой последовательности с искомым значением v и сразу исключить половину последовательности из дальнейшего рассмотрения. **Бинарный поиск** (binary search) - это алгоритм, в котором такая процедура повторяется неоднократно, что свякий раз приводит к уменьшению осташейся части последовательности в два раза. Запишите псевдокод алгоритма бинраного поиска (либо итеративный, либо рекурсивный). Докажите, что время работы этого алгоритма в наихудшем случае составялет $\Theta(\lg n)$.

Полагаем массив A содержит k.

Псевдокод:

```
Binary-Search(A, k)
  l = 0
  r = A.length + 1
  while l < r − 1
    m = ⌊(l + r)/2⌋
    if A[m] < k
      l = m
    else
      r = m
  if r ≤ A.length and A[r] == k
    return r
  else
    return NIL
```

Доказательство: В наихудшем случае искомый элемент отсутствует в массиве или находится на последней проверяемой позиции. На каждой итерации цикла область поиска уменьшается примерно вдвое: после первой итерации остаётся не более $\lceil n/2 \rceil$ элементов, после второй — не более $\lceil n/4 \rceil$, и так далее. Через k итераций размер области поиска не превышает $\lceil n/2^k \rceil$. Процесс продолжается, пока область поиска не станет пустой, т.е. $n/2^k < 1$. Отсюда $k > \log_2 n$, то есть количество итераций равно $\lfloor \log_2 n \rfloor + 1 = \Theta(\log n)$. Поскольку каждая итерация выполняется за $\Theta(1)$, общее время работы в наихудшем случае составляет $\Theta(\log n)$.

2.3.6 Заметим, что в цикле **while** в строках 5-7 процедуры Insertion-Sort в раделе 2.1 для сканирования (в обратном порядке) отсортированного помассива $A[1 \ldots j-1]$ используется линейный поиск. Можно ли использовать бинарный поиск (см. упр. 2.3.5) вместо линейного поиска, чтобы время работы этого алгоритма в наихудшем случае улучшилось и стало $\Theta(n \lg n)$?

Задачи

2.1. Сортировка вставкой малых массивов в процессе сортировки слиянием Несмотря на то что с увеличением количества сортируемых элементов время сортировки методом слияния в наихудшем случае растёт как $\Theta(n \lg n)$, а время сортировки вставками — как $\Theta(n^2)$, благодаря постоянным множителям на практике для малых размеров задач на большинстве машин сортировка вставками выполняется быстрее. Таким образом, имеет смысл использовать сортировку вставками в

процессе сортировки методом слияния, когда подзадачи становятся достаточно маленькими. Рассмотрим модификацию алгоритма сортировки слиянием, в котором n/k подмассивов длиной k сортируются вставками, после чего они объединяются с помощью обычного механизма слияния. Величина k должна быть найдена в процессе решения задачи.

а. Покажите, что сортировка вставками позволяет отсортировать n/k подпоследовательностей длиной k каждая за время $\Theta(nk)$ в худшем случае.

   Ответ: В наихудшем случае сортировка вставками сортирует подпоследовательность длиной k за время $\Theta(k^2)$. Поскольку таких подпоследовательностей n/k, общее время сортировки вставками составит $\Theta(nk)$.

б. Покажите, как выполнить слияние этих подпоследовательностей за время $\Theta(n \lg(n/k))$ в наихудшем случае.

   Для одного уровня слияния требуется время $\Theta(n)$. Если построить дерево рекурсии, то на каждом уровне общее количество объединяемых элементов равно n. Количество уровней равно $\Theta(\lg(n/k))$. Следовательно, общее время слияния составляет $\Theta(n \lg(n/k))$.

в. Если такой модифицированный алгоритм выполняется за время $\Theta(nk + n \lg(n/k))$ в наихудшем случае, то чему равно наибольшее значение k как функции от n, при котором модифицированный алгоритм в Θ-обозначениях имеет то же время работы, что и стандартная сортировка слиянием?

   Для этого приравняем времена работы алгоритмов:

$$c_1(nk + n \lg(n/k)) = c_2 n \lg n$$

$$c_1 k + c_1 \lg(n/k) = c_2 \lg n$$

$$c_1 k + c_1 \lg n - c_1 \lg k = c_2 \lg n$$

$$c_1 k - c_1 \lg k = c_2 \lg n - c_1 \lg n$$

$$c_1(k - \lg k) = (c_2 - c_1) \lg n$$

   Из равенства

$$k - \lg k = \Theta(\lg n)$$

   заметим, что $\lg k = o(k)$, поэтому

$$k - \lg k = \Theta(k)$$

Следовательно,

$$k = \Theta(\lg n).$$

г. Как следует выбирать k на практике?

На практике выбор k можно осуществлять на основе эмпирических данных, полученных при замерах времени работы алгоритмов. Можно найти такое значение n, при котором сортировка слиянием и вставками показывает одинаковое время работы, и использовать значение, немного меньшее найденного, в качестве порогового для переключения на сортировку вставками.

2.2. Корректность пузырьковой сортировки Пузырьковая сортировка представляет собой популярный, но неэффективный алгоритм сортировки. В его основе лежит многократная перестановка соседних элементов, нарушающих порядок сортировки.

```
BUBBLESORT(A)
1  for i = 1 to A.length - 1
2      for j = A.length downto i + 1
3          if A[j] < A[j - 1]
4              поменять A[j] и A[j - 1] местами
```

а. Пусть A' обозначает выход процедуры BUBBLESORT(A). Для доказательства корректности процедуры BUBBLESORT необходимо доказать, что она завершается и что

$$A'[1] \leq A'[2] \leq \cdots \leq A'[n],$$

где $n = A$.length. Что ещё необходимо доказать для того, чтобы показать, что процедура BUBBLESORT действительно выполняет сортировку?

Ответ:
Для подтверждения корректности алгоритма необходимо также доказать, что результирующий массив A' является перестановкой исходного массива A.

В следующих двух частях доказывается неравенство выше.

б. Точно сформулируйте инвариант цикла `for` в строках 2–4 и докажите, что он выполняется.

Инвариант цикла:
На каждой итерации цикла при текущем значении j элемент $A[j]$ является наименьшим среди элементов $A[j \dots n]$.

Инициализация:
При $j = A.\text{length}$ подмассив $A[j \dots n]$ содержит один элемент $A[j]$, который является наименьшим.

Сохранение:
Предположим, что перед итерацией при текущем значении j элемент $A[j]$ является наименьшим среди $A[j \dots n]$. Сравниваются элементы $A[j]$ и $A[j-1]$, и при необходимости выполняется обмен, в результате чего на позиции $j-1$ оказывается наименьший элемент из $A[j-1 \dots n]$. После уменьшения j инвариант сохраняется.

Завершение:
При завершении цикла $j = i$, и по инварианту элемент $A[i]$ является наименьшим среди элементов $A[i \dots n]$.

в. Сформулируйте инвариант внешнего цикла.

Инвариант цикла:
На каждой итерации цикла `for` в строках 1–4 подмассив $A[1 \dots i-1]$ содержит наименьшие элементы массива и отсортирован по возрастанию.

Инициализация:
При $i = 1$ подмассив $A[1 \dots i-1]$ пуст, следовательно, он тривиально отсортирован.

Сохранение:
Предположим, что перед итерацией при текущем значении i подмассив $A[1 \dots i-1]$ содержит наименьшие элементы массива и отсортирован. Внутренний цикл перемещает наименьший элемент из подмассива $A[i \dots n]$ на позицию i. Таким образом, после увеличения i подмассив $A[1 \dots i]$ содержит наименьшие элементы и остаётся отсортированным.

Завершение:
При завершении цикла $i = n$, подмассив $A[1 \dots n]$ содержит все элементы массива в отсортированном порядке.

г. Определите время работы пузырьковой сортировки в наихудшем случае.

Ответ:
Время работы можно оценить следующим образом:

$$\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2} = \Theta(n^2).$$

2.3. Корректность правила Горнера Следующий фрагмент кода реализует правило Горнера для вычисления полинома

$$\begin{aligned} P(x) &= \sum_{k=0}^{n} a_k x^k \\ &= a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-1} + xa_n)\ldots)) \end{aligned}$$

для заданных коэффициентов $a_0, a_1, \ldots, a_n$ и значения x.

```
y = 0
for i = n downto 0
    y = a_i + x · y
```

а. Чему равно время работы этого фрагмента кода правила Горнера в Θ-обозначениях?

Ответ:
$\Theta(n)$

б. Напишите псевдокод, реализующий алгоритм обычного вычисления полинома, когда каждое слагаемое полинома вычисляется отдельно. Определите асимптотическое время работы этого алгоритма и сравните его со временем работы алгоритма, основанного на правиле Горнера.

Псевдокод:

```
y = 0
for k = 0 to n
    power = 1
    for i = 1 to k
        power = power · x
    y = y + a_k · power
```

Ответ: Время работы алгоритма можно выразить:

$$\sum_{k=0}^{n}\left(\sum_{i=1}^{k} 1 + 1\right) = \sum_{k=0}^{n}(k+1) = \Theta(n \cdot k) = \Theta(n^2)$$

Вывод: Асимптотическое время работы наивного решения является квадратичной функцией от n, алгоритм основанный на правиле Горнера выполняется за линейное время.

в. Рассмотрим следующий инвариант цикла.

В начале каждой итерации цикла **for** в строках 2 и 3

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k.$$

Рассматривайте сумму без членов как равную нулю. Следуя структуре доказательства инварианта цикла, которая использовалась ранее в данной главе, воспользуйтесь указанным инвариантом цикла, чтобы показать, что по завершении работы $y = \sum_{k=0}^{n} a_k x^k$.

Инициализация:

Рассмотрим значение при $i = n$:

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k = \sum_{k=0}^{-1} a_{k+n+1} x^k = 0$$

Что соотвествует значению y перед началом работы цикла 2-3.

Сохранение:

Допустим инариант корретный, тогда для $i = m$:

$$y_m = \sum_{k=0}^{n-(m+1)} a_{k+m+1} x^k$$

что соотвествует $y_m = a_m + x \cdot y_{m-1}$ по алгоритму.

Тогда для $m = m + 1$:

$$y_{m+1} = a_m + x \cdot \sum_{k=0}^{n-(m+1)} a_{k+m+1} x^k = a_m + \sum_{k=0}^{n-(m+1)} a_{k+m+1} x^{k+1}$$

Введём новую переменную $j = k + 1$. Тогда $k = j - 1$, и когда k пробегает от 0 до $n - (m + 1)$, j пробегает от 1 до $n - m$:

$$y_{m+1} = a_m + \sum_{j=1}^{n-m} a_{j+m} x^j$$

Заметим что $a_m = a_{j+m}x^j$ при $j = 0$

$$y_{m+1} = \sum_{j=0}^{n-m} a_{j+m}x^j$$

Завершение:

г. Сделайте заключение, что в приведенном фрагменте кода правильно вычисляется значение полинома, который задается коэффициентами $a_0, a_1, \ldots, a_n$.

2.4. Инверсии Пусть $A[1 \ldots n]$ представляет собой массив из n различных чисел. Если $i < j$ и $A[i] > A[j]$, то пара (i, j) называется **инверсией** A.

а. Перечислите пять инверсий массива $\langle 2, 3, 8, 6, 1 \rangle$.

Ответ: $(1, 5), (2, 5), (3, 4), (3, 5), (4, 5)$

б. Какой массив из элементов множества $\{1, 2, \ldots, n\}$ содержит максимальное количество инверсий? Сколько инверсий в этом массиве?

Ответ: Максимальное кол-во инверсий содержит отсортированный по убыванию массив $\{n, n-1, \ldots, 1\}$ и содержит:

$$\sum_{i=1}^{n-1} (n-i) = \sum_{i=1}^{n} (n-i) = n^2 - \frac{n \cdot (n+1)}{2} = \frac{n \cdot (n-1)}{2}$$

в. Какая существует взаимосвязь между временем сортировки методом вставок и количеством инверсий во входном массиве? Обоснуйте свой ответ.

г. Разработайте алгоритм, определяющий количество инверсий, содержащихся в произвольной перестановке n элементов, время работы которого в наихудшем случае равно $\Theta(n \lg n)$. (*Указание:* модифицируйте алгоритм сортировки слиянием.)

4 Глава 3. Рост функции

3.1. асимптотические обозначения

3.1.1 Пусть $f(n)$ и $g(n)$ — асимптотически неотрицательные функции. Докажите с помощью базового определения Θ-обозначений, что

$$\max(f(n), g(n)) = \Theta(f(n) + g(n)).$$

Ответ:

Заметим, что для любых неотрицательных функций $f(n)$ и $g(n)$ выполняются следующие неравенства:

$$\max(f(n), g(n)) \leq f(n) + g(n),$$

так как максимум не превышает сумму.

С другой стороны,

$$f(n) \leq \max(f(n), g(n)), \quad g(n) \leq \max(f(n), g(n)),$$

следовательно,

$$f(n) + g(n) \leq 2 \cdot \max(f(n), g(n)).$$

Отсюда получаем:

$$\frac{1}{2}(f(n) + g(n)) \leq \max(f(n), g(n)) \leq f(n) + g(n).$$

Таким образом, существуют положительные константы $c_1 = \frac{1}{2}$ и $c_2 = 1$, такие что

$$c_1(f(n) + g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n)).$$

По определению Θ-обозначения получаем:

$$\max(f(n), g(n)) = \Theta(f(n) + g(n)).$$

3.1.2 Покажите, что для любых действительных констант a и b, где $b > 0$, выполняется соотношение

$$(n + a)^b = \Theta(n^b).$$

Ответ:

Рассмотрим предел:

$$\lim_{n\to\infty} \frac{(n + a)^b}{n^b} = \lim_{n\to\infty} \left(1 + \frac{a}{n}\right)^b = 1^b = 1.$$

По определению предела, для $\varepsilon = \frac{1}{2}$ найдётся N, такое что для всех $n \geq N$:

$$\frac{1}{2} \leq \left(1 + \frac{a}{n}\right)^b \leq \frac{3}{2}.$$

Умножая на n^b, получаем:

$$\frac{1}{2}n^b \leq (n+a)^b \leq \frac{3}{2}n^b.$$

Следовательно, $(n+a)^b = \Theta(n^b)$ с константами $c_1 = \frac{1}{2}$, $c_2 = \frac{3}{2}$ (для всех $n \geq N$). Для начальных n константы можно при необходимости увеличить.

3.1.3 Поясните, почему утверждение «время работы алгоритма A равно как минимум $O(n^2)$» лишено смысла.

Ответ:

$O(n^2)$ обозначает **верхнюю** границу: $T(n) \leq c \cdot n^2$ для достаточно больших n. Фраза «как минимум» требует **нижней** границы. Смешивать их некорректно, так как $O(n^2)$ включает функции, растущие медленнее n^2 (например, n или $\log n$), и утверждение «$T(n) \geq$ что-то из $O(n^2)$» не накладывает содержательного ограничения снизу. Правильно: $\Omega(n^2)$ для нижней границы.

3.1.4 Справедливы ли соотношения

$$2^{n+1} = O(2^n)$$

и

$$2^{2n} = O(2^n)?$$

Ответ:

Первое справедливо:

$$2^{n+1} = 2 \cdot 2^n = O(2^n),$$

так как существует константа $c = 2$ и $n_0 \geq 1$, при которых $2 \cdot 2^n \leq c \cdot 2^n$.

Второе не справедливо. Предположим противное: $2^{2n} = O(2^n)$. Тогда по определению $\exists c, n_0 : \forall n \geq n_0$ выполнено $2^{2n} \leq c \cdot 2^n$. Сокращая на 2^n, получаем $2^n \leq c$, что невозможно при $n \to \infty$. Следовательно, соотношение не выполняется.

3.1.5 Докажите теорему 3.1.

3.1.6 Докажите, что время работы алгоритма равно $\Theta(g(n))$ тогда и только тогда, когда его время работы в наилучшем случае равно $O(g(n))$, а в наихудшем — $\Omega(g(n))$.

3.1.7 Докажите, что множество $o(g(n)) \cap \omega(g(n))$ является пустым.
Ответ:

Предположим противное: пусть существует функция $f(n) \in o(g(n)) \cap \omega(g(n))$.

По определению $f(n) = \omega(g(n))$: для любой константы $c > 0$ найдётся n_1, что при всех $n \geq n_1$ выполняется $f(n) > c \cdot g(n)$.

По определению $f(n) = o(g(n))$: для любой константы $c > 0$ найдётся n_2, что при всех $n \geq n_2$ выполняется $f(n) < c \cdot g(n)$.

Возьмём, например, $c = 1$. Тогда для достаточно больших n (начиная с $\max(n_1, n_2)$) одновременно:

$$f(n) > 1 \cdot g(n) \quad \text{и} \quad f(n) < 1 \cdot g(n).$$

Получили противоречие: $f(n)$ не может быть одновременно строго больше и строго меньше $g(n)$.

Следовательно, такой функции не существует, и пересечение пусто.

3.1.8 Можно обобщить наши обозначения на случай двух параметров n и m, которые могут возрастать до бесконечности по отдельности с разными скоростями. Для данной функции $g(n, m)$ обозначим как $O(g(n, m))$ множество функций

$$O(g(n,m)) = \{f(n,m) : \text{существуют положительные константы } c, n_0 \text{ и } m_0, \text{ такие, что } 0 \leq f(n,$$

для всех $n \geq n_0$ или $m \geq m_0$.